



A sub-linear time algorithm for approximating k-nearest-neighbor with full quality guarantee [☆]

Hengzhao Ma, Jianzhong Li ^{*}

Harbin Institute of Technology, Harbin, Heilongjiang 150001, China

ARTICLE INFO

Article history:

Received 26 November 2020

Received in revised form 14 December 2020

Accepted 22 December 2020

Available online 29 December 2020

Keywords:

Computation geometry

Approximate k-nearest-neighbors

ABSTRACT

In this paper we propose an algorithm for the approximate k-Nearest-Neighbors problem. According to the existing researches, there are two kinds of approximation criteria. One is the distance criterion, and the other is the recall criterion. All former algorithms suffer the problem that there are no theoretical guarantees for the two approximation criteria. The algorithm proposed in this paper unifies the two kinds of approximation criteria, and has full theoretical guarantees. Furthermore, the query time of the algorithm is sub-linear. As far as we know, it is the first algorithm that achieves both sub-linear query time and full theoretical approximation guarantee.

© 2020 Published by Elsevier B.V.

1. Introduction

The k-Nearest-Neighbor (kNN) problem is a well-known problem in theoretical computer science and applications. Let (U, D) be a metric space, then given a set of elements $P \subseteq U$ and a query element $q \in U$, the kNN problem is to find the k elements with smallest distance to q . However, the exact results are expensive to compute when the size of the input is large [21,37], and approximate results serve as good as the exact ones in many applications, such as data usability [10,27,29], database queries [26], and graph algorithms [28]. Thus, the approximate kNN, kANN for short, draws more research efforts in recent years. There are two kinds of approximation criteria for the kANN problem, namely, the distance criterion and the recall criterion. The distance criterion requires that the ratio between the distance from the approximate results to the query and the distance from the exact results to the query is no more than a given threshold. The recall criterion requires that the size of the intersection of the approximate result set and the exact result set is no less than a given threshold. The formal description will be given in detail in Section 2. Next we brief the existing algorithms for the kANN problem to see how these two criteria are considered by former researchers.

The algorithms for the kANN problem can be categorized into four classes. The first class is the tree-based methods. The main idea of this method is to recursively partition the metric space into sub-spaces, and organize them into a tree structure. The K-D tree [6] is the representative idea in this category. It is efficient in low dimensional spaces, but the performance drops rapidly when the number of dimension grows up. Vantage point tree (VP-tree) [38] is another data structure with a better partition strategy and better performance. The FLANN [32] method is a recent work with improved performance in high dimensional spaces, but it is reported that this method would achieve in sub-optimal results [22]. To the best of our knowledge, the tree based methods can satisfy neither the distance nor the recall criterion theoretically.

[☆] This work was supported by the National Natural Science Foundation of China under grant 61732003, 61832003, 61972110 and U1811461.

^{*} Corresponding author.

E-mail addresses: hzma@stu.hit.edu.cn (H. Ma), lijzh@hit.edu.cn (J. Li).

The second class is the permutation based methods. The idea is to choose a set of pivot points, and represent each data element by a permutation of the pivots, which is decided by sorting the pivots according to the distance to the element. In such a representation, close elements will have similar permutations. Methods using the permutation idea include the MI-File [2] and PP-Index [14]. Unfortunately, the permutation based method can not satisfy either of the distance or the recall criterion theoretically, as far as we know.

The third class is the Locality Sensitive Hashing (LSH) based methods. LSH was first introduced by Indyk et al. [21] for the kANN problem where $k = 1$. Soon after, Datar et al. [12] proposed the first practical LSH function, and since then there came a burst in the theoretical and applicational researches on the LSH framework [17,23]. For example, Andoni et al. proved the lower bound of the time-space complexities of the LSH based algorithms [3], and devised the optimal LSH function which meets the lower bound [4]. On the other hand, Gao et al. [16] proposed an algorithm that aimed to close the gap between the LSH theory and kANN search applications. See [36] for a survey. The basic LSH based method can satisfy only the distance criterion when $k = 1$ [21]. Some existing algorithms made some progress. The C2LSH algorithm [15] solved the kANN problem with the distance criterion, but it has a constraint that the approximation factor must be a square of an integer. The SRS algorithm [35] is another one aimed at the distance criterion. However, it only has partial guarantee, that is, the results satisfy the distance criterion only when the algorithm terminates on a specific condition.

The fourth class is graph based methods. The specific kind of graphs used in this method is the proximity graphs, where the edges in this kind of graph are defined by the geometric relationship of the points. See [30] for a survey. The graph based kANN algorithms usually conduct a navigating process on the proximity graphs. This process selects an vertex in the graph as the start point, and move to the destination point following some specific navigating strategy. For example, Paredes et al. [34] used the kNN graph, Ocsa et al. [33] used the Relative Neighborhood Graph (RNG), and Malkov et al. [25] used the Navigable Small World Graph (NSW). None of these algorithms have theoretical guarantee on the two approximation criteria.

In summary, most of the existing algorithms do not have theoretical guarantee on either of the two approximation criteria. The recall criterion is only used as a measurement of the experimental results, and the distance criterion is only partially satisfied by only a few algorithms [15,35]. In this paper, we propose a sub-linear time algorithm for kANN problem that unifies the two kinds of approximation criteria, which overcomes the disadvantages of the existing algorithms. The contributions of this paper are listed below.

1. We propose an algorithm that unifies the distance criterion and the recall criterion for the approximate k-Nearest-Neighbor problem. The result returned by the algorithm can satisfy at least one criterion in any situation. Further analysis shows that the returned result may as well satisfy both of the two criteria with a certain probability. This is a major progress compared to the existing algorithms.
2. Assuming the input point set follows the Poisson Point Process, the algorithm takes $O(n \log n)$ time of preprocessing, $O(n \log n)$ space, and answers a query in $O(dn^{1/d} \log n + kn^\rho \log n)$ time, where $\rho < 1$ is a constant.
3. The algorithm is the first algorithm for kANN that provides theoretical guarantee on both of the approximation criteria, and it is also the first algorithm that achieves sub-linear query time while providing theoretical guarantees. The former works [15,35] with partial guarantee both need linear query time.

The rest of this paper is organized as follows. Section 2 introduces the definition of the problem and some prerequisite knowledge. The detailed algorithms are presented in Section 3. Then the correctness and complexities are analyzed in Section 4. Finally the conclusion is given in Section 5.

2. Preliminaries

2.1. Problem definitions

The problem studied in this paper is the approximate k-Nearest-Neighbor problem, which is denoted as kANN for short. In this paper the problem is constrained to the Euclidean space. The input is a set P of points where each $p \in P$ is a d -dimensional vector $(p^{(1)}, p^{(2)}, \dots, p^{(d)})$. The distance between two points p and p' is defined by $D(p, p') = \sqrt{\sum_{i=1}^d (p^{(i)} - p'^{(i)})^2}$, which is the well known Euclidean distance. Before giving the definition of the kANN problem, we first introduce the exact kNN problem.

Definition 2.1 (kNN). Given the input point set $P \subset R^d$ and a query point $q \in R^d$, define $kNN(q, P)$ to be the set of k points in P that are nearest to q . Formally,

1. $kNN(q, P) \subseteq P$, and $|kNN(q, P)| = k$;
2. $D(p, q) \leq D(p', q)$ for $\forall p \in kNN(q, P)$ and $\forall p' \in P \setminus kNN(q, P)$.

Next we will give the definition of the approximate kNN. There are two kinds of definitions based on different approximation criteria.

Definition 2.2 ($kANN_c$). Given the input point set $P \subset R^d$, a query point $q \in R^d$, and a approximation factor $c > 1$, find a point set $kANN_c(q, P)$ which satisfies:

1. $kANN_c(q, P) \subseteq P$, and $|kANN_c(q, P)| = k$;
2. let $T_k(q, P) = \max_{p \in kNN(q, P)} D(p, q)$, then $D(p', q) \leq c \cdot T_k(q, P)$ holds for $\forall p' \in kANN_c(q, P)$.

Remark 2.1. The second requirement in Definition 2.2 is called the distance criterion.

Definition 2.3 ($kANN_\delta$). Given the input point set $P \subset R^d$, a query point $q \in R^d$, and a approximation factor $\delta < 1$, find a point set $kANN_\delta(q, P) \subseteq P$ which satisfies:

1. $kANN_\delta(q, P) \subseteq P$, and $|kANN_\delta(q, P)| = k$;
2. $|kANN_\delta(q, P) \cap kNN(q, P)| \geq \delta \cdot k$.

Remark 2.2. If a kANN algorithm returned a set S , the value $\frac{|S \cap kNN(q, P)|}{|kNN(q, P)|}$ is usually called the recall of the set S . This is widely used in many works to evaluate the quality of the kANN algorithm. Thus we call the second statement in Definition 2.3 as the recall criterion.

Next we give the definition of the problem studied in this paper, which unifies the two different criteria.

Definition 2.4. Given the input point set $P \subset R^d$, a query point $q \in R^d$, and approximation factors $c > 1$ and $\delta < 1$, find a point set $kNN_{c,\delta}(q, P)$ which satisfies:

1. $kANN_{c,\delta}(q, P) \subseteq P$, and $|kANN_{c,\delta}(q, P)| = k$;
2. $kANN_{c,\delta}(q, P)$ satisfies at least one of the distance criterion and the recall criterion. Formally, either $D(p', q) \leq c \cdot T_k(q, P)$ holds for $\forall p' \in kANN_{c,\delta}(q, P)$, or $|kANN_{c,\delta}(q, P) \cap kNN(q, P)| \geq \delta \cdot k$.

According to Definition 2.4, the output of the algorithm is required to satisfy at least one of the two criteria, but not both. It will be our future work to devise an algorithm to satisfy both of the criteria.

In the rest of this section we will introduce some concepts and algorithms that will be used in our proposed algorithm.

2.2. Minimum enclosing spheres

The d-dimensional spheres are the generalization of the circles in the 2-dimensional case. Let c be the center and r be the radius. A d-dimensional sphere, denoted as $S(c, r)$, is the set $S(c, r) = \{x \in R^d \mid D(x, c) \leq r\}$. Note that the boundary is included. If $q \in S(c, r)$ we say that q falls inside sphere $S(c, r)$, or the sphere encloses point p . A sphere $S(c, r)$ is said to pass through point p iff $D(c, p) = r$.

Given a set P of points, the minimum enclosing sphere (MES) of P , is the d-dimensional sphere enclosing all points in P and has the smallest possible radius. It is known that the MES of a given finite point set in R^d is unique, and can be calculated by a quadratic programming algorithm [39]. Next we introduce the approximate minimum enclosing spheres.

Definition 2.5 (AMES). Given a set of points $P \subset R^d$ and an approximation factor $\epsilon < 1$, the approximate minimum enclosing sphere of P , denoted as $AMES(P, \epsilon)$, is a d-dimensional sphere $S(c, r)$ satisfies:

1. $p \in S(c, r)$ for $\forall p \in P$;
2. $r < (1 + \epsilon)r^*$, where r^* is the radius of the exact MES of P .

The following algorithm can calculate the AMES in $O(n/\epsilon^2)$ time, which is given in [5].

Algorithm 1: Compute AMES.

Input: a point set P , and an approximation factor ϵ .

Output: $AMES(P, \epsilon)$

```

1  $c_0 \leftarrow$  an arbitrary point in  $P$ ;
2 for  $i = 1$  to  $1/\epsilon^2$  do
3    $p_i \leftarrow$  the point in  $P$  farthest away from  $c_{i-1}$ ;
4    $c_i \leftarrow c_{i-1} + \frac{1}{i}(p_i - c_{i-1})$ ;
5 end
```

The following Lemma gives the complexity of Algorithm 1.

Lemma 2.1 ([5]). For given ϵ and P where $|P| = n$, Algorithm 1 can calculate $AMES(P, \epsilon)$ in $O(n/\epsilon^2)$ time.

2.3. Delaunay triangulation

The Delaunay Triangulation (DT) is a fundamental data structure in computation geometry. The definition is given below.

Definition 2.6 (DT). Given a set of points $P \subset \mathbb{R}^d$, the Delaunay Triangulation is a graph $DT(P) = (V, E)$ which satisfies:

1. $V = P$;
2. for $\forall p, p' \in P$, $(p, p') \in E$ iff there exists a d -dimensional sphere passing through p and p' , and no other $p'' \in P$ is inside it.

The Delaunay Triangulation is a natural dual of the Voronoi diagram. We omit the details about their relationship since it is not the focus of this paper.

There are extensive research works about the Delaunay triangulation. An important problem is to find the expected properties of DT built on random point sets. Here we focus on the point sets that follow the Poisson Point Process in d -dimensional Euclidean space. In this model, for any region $\mathcal{R} \subset \mathbb{R}^d$, the probability that $\mathcal{R}$ contains k points follows the Poisson distribution. See [1] for more details. We cite one important property of the Poisson Point Process in the following lemma.

Lemma 2.2 ([1]). Let $S \subset \mathbb{R}^d$ be a point set following the Poisson Point Process. Suppose there are two regions $B \subseteq A \subset \mathbb{R}^d$. For any point $p \in S$, if p falls inside A then the probability that p falls inside B is the ratio between the volume of B and A . Formally, we have

$$\Pr[p \in B \mid p \in A] = \frac{\text{volume}(B)}{\text{volume}(A)}.$$

Further, we cite some important properties of the Delaunay triangulation built on point sets which follow the Poisson Point Process.

Lemma 2.3 ([7]). Let $S \subset \mathbb{R}^d$ be a point set following the Poisson Point Process, and $\Delta(G) = \max_{p \in V(G)} |\{(p, q) \in E(G)\}|$ be the maximum degree of G . Then the expected maximum degree of $DT(S)$ is $O(\log n / \log \log n)$.

Lemma 2.4 ([9]). Let $S \subset \mathbb{R}^d$ be a point set following the Poisson Point Process. The expected time to construct $DT(S)$ is $O(n \log n)$.

2.4. Walking in Delaunay triangulation

Given a Delaunay Triangulation DT , the points and edges of DT form a set of simplices. Given a query point q , there is a problem to find which simplex of DT that q falls in. There is a class of algorithms to tackle this problem which is called Walking. The Walking algorithm starts at some simplex, and walk to the destination by moving to adjacent simplices step by step. There are several kinds of walking strategy, including Jump & Walk [31], Straight Walk [8] and Stochastic Walk [13], etc. Some of these strategies are only applicable to 2 or 3 dimension, while Straight Walk can generalize to higher dimensions. As Fig. 1 shows, the Straight Walk strategy only considers the simplices that intersect the line segment from the start point to the destination. The following lemma gives the complexity of this walking strategy.

Lemma 2.5 ([11]). Given a Delaunay Triangulation DT of a point set $P \subset \mathbb{R}^d$, and two points p and q in $\mathbb{R}^d$ as the start point and destination point, the walking from p to q using Straight Walk takes $O(n^{1/d})$ expected time.

2.5. (c, r) -NN

The Approximate Near Neighbor problem is introduced in [21] for solving the $kANN_c$ problem with $k = 1$. Usually the Approximate Near Neighbor problem is denoted as (c, r) -NN since there are two input parameters c and r . The definition is given below. The idea to use (c, r) -NN to solve $1ANN_c$ is via Turing reduction, that is, use (c, r) -NN as an oracle or sub-procedure. The details can be found in [19–21,24].

Definition 2.7. Given a point set P , a query point q , and two query parameters $c > 1, r > 0$, the output of the (c, r) -NN problem should satisfy:

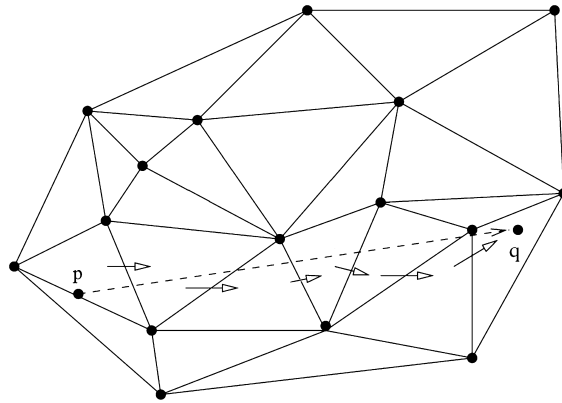


Fig. 1. Illustration of the Straight Walk.

1. if $\exists p^* \in S(q, r) \cap P$, then output a point $p' \in S(q, c \cdot r) \cap P$;
2. if $D(p, q) > c \cdot r$ for $\forall p \in P$, then output No;

Since we aim to solve kANN problem in this paper, we need the following definition of (c, r) -kNN.

Definition 2.8. Given a point set P , a query point q , and two query parameters $c > 1, r > 0$, the output of the (c, r) -kNN problem is a set $kNN_{(c,r)}(q, P)$, which satisfies:

1. if $|P \cap S(q, r)| \geq k$, then output a set $Q \subseteq P \cap S(q, c \cdot r)$, where $|Q| = k$;
2. if $|P \cap S(q, c \cdot r)| < k$, then output $\emptyset$;

Corollary 2.1. Denote Res to be the returned set of (c, r) -kNN, then we have the following claim.

1. If $Res = \emptyset$, then it must be the case that $|P \cap S(q, r)| < k$.
2. If $Res \neq \emptyset$, then it must be the case that $|P \cap S(q, cr)| \geq k$.

It can be easily seen that the (c, r) -kNN problem is a natural generalization of the (c, r) -NN problem. Recently, there are several algorithms proposed to solve this problem. The following Lemma 2.6 gives the complexity of the (c, r) -kNN algorithm, which will be proved in Appendix A.

Lemma 2.6. There is an algorithm that solves (c, r) -kNN problem in $O(kn^\rho)$ of time, requiring $O(kn^{1+\rho} \log n)$ time of preprocessing and $O(kn^{1+\rho})$ of space. The parameter ρ is a constant depending on the LSH function used in the algorithm, and $\rho < 1$ always holds.

3. Algorithm

The proposed algorithm consists of two phases, i.e., the preprocessing phase and the query phase. The preprocessing phase is to build a data structure, which will be used to guide the search in the query phase. Next we will describe the algorithm of the two phases in detail.

3.1. Preprocessing algorithm

Before describing the details of the preprocessing algorithm, we first introduce several concepts that will be used in the following discussion.

Axis Parallel Box. An axis parallel box B in R^d is defined to be the Cartesian product of d intervals, i.e., $B = I_1 \times I_2 \times \cdots \times I_d$. And the following is the definition of Minimum Bounding Box.

Definition 3.1. Given a point set P , the Minimum Bounding Box, denoted as $MBB(P)$, is the axis parallel box satisfying the following two requirements:

1. $MBB(P)$ encloses all points in P ;
2. there exists points p and p' in P such that $p^{(i)} = a_i, p'^{(i)} = b_i$ for each interval $I_i = (a_i, b_i)$ defining $MBB(P)$, $1 \leq i \leq d$.

Median Split. Given a point set P and its minimum bounding box $MBB(P)$, we introduce an operation on P that splits P into two subsets, which is called median split. This operation first finds the longest interval I_i from the intervals defining $MBB(P)$. Then, the operation finds the median of the set $\{p^{(i)} \mid p \in P\}$, which is the median of the i -th coordinates of the points in P . This median is denoted as $med_i(P)$. Finally P is split into two subsets, i.e., $P_1 = \{p \in P \mid p^{(i)} \leq med_i(P)\}$ and $P_2 = \{p \in P \mid p^{(i)} > med_i(P)\}$. Here we assume that no two points share the same coordinate in any dimension. This assumption can be assured by adding some random small shift on the original coordinates.

Median Split Tree. By recursively conducting the median split operation, a point set P can be organized into a tree structure, which is called the Median Split Tree (MST). The definition of MST is given below.

Definition 3.2. Given the input point set P , a Median Split Tree (MST) based on P , denoted as $MST(P)$, is a tree structure satisfying the following requirements:

1. the root of $MST(P)$ is P , and the other nodes in $MST(P)$ are subsets of P ;
2. there are two child nodes for each interior node $N \in MST(P)$, which are generated by conducting a median split on N ;
3. each leaf node contains only one point.

Balanced Median Split Tree. The depth of a node N in a tree T , denoted as $dep_T(N)$, is defined to be the number of edges in the path from N to the root of T . It can be noticed that the leaf nodes in the MST may have different depths. So we introduce the Balanced Median Split Tree (BMST), where all leaf nodes have the same depth.

Let $L_T(i) = \{N \in T \mid dep_T(N) = i\}$, which is the set of nodes in the i -th layer in tree T , and $|N|$ be the number of points included in node N . For a median split tree $MST(P)$, it can be easily proved that either $|N| = \lceil n/2^i \rceil$ or $|N| = \lfloor n/2^i \rfloor$ for $\forall N \in L_{MST(P)}(i)$. Given $MST(P)$, the $BMST(P)$ is constructed as follows. Find the smallest i such that $\lfloor n/2^i \rfloor \leq 3$, then for each node $N \in L_{MST(P)}(i)$, all the nodes in the sub-tree rooted at N are directly connected to N .

Hierarchical Delaunay Graph. Given a point set P , we introduce the most important concept for the preprocessing algorithm in this paper, which is the Hierarchical Delaunay Graph (HDG). This structure is constructed by adding edges between nodes in the same layer of $BMST(P)$. The additional edges are called the graph edges, in contrast with the tree edges in $BMST(P)$. The definition of the HDG is given below. Here $Cen(N)$ denotes the center of $AMES(N)$.

Definition 3.3. Given a point set P and the balanced median split tree $BMST(P)$, a Hierarchical Delaunay Graph HDG is a layered graph based on $BMST(P)$, where each layer is a Delaunay triangulation. Formally, for each $N, N' \in HDG(P)$, there is an graph edge between N, N' iff

1. $dep_{BMST(P)}(N) = dep_{BMST(P)}(N')$, and
2. there exists a d -dimensional sphere S passing through $Cen(N), Cen(N')$, and there is no $N'' \in HDG(P)$ such that $Cen(N'')$ falls in S , where N'' is in the same layer with N and N' . That is, the graph edges connecting nodes in the same layer form the Delaunay Triangulation.

The Preprocessing Algorithm. Next we describe the preprocessing algorithm which aims to build the HDG. The algorithm can be divided into three steps.

Step 1, split and build tree. This step is to recursively split P into smaller sets using the median split operation, and build the median split tree. Finally the nodes near the leaf layer are adjusted to satisfy the definition of the balanced median split tree.

Step 2, compute spheres. In this step, the algorithm will go over the tree and compute the AMES for each node using Algorithm 1.

Step 3, construct the HDG . In this step, an algorithm given in [9] which satisfies Lemma 2.4 is invoked to compute the Delaunay triangulation for each layer.

The pseudo codes of the preprocessing algorithm are given in Algorithm 2.

3.2. Query algorithm

The query algorithm takes the HDG built by the preprocessing algorithm, and executes the following three steps.

The first is the descending step. The algorithm goes down the tree and stops at level i such that $k \leq n/2^i < 2k$. At each level, the child node with smallest distance to the query is chosen to be visited in next level.

The second is the navigating step. The algorithm marches towards the local nearest AMES center by moving on the edges of the HDG .

The third step is the answering step. The algorithm finds the answer of $kANN_{c,\delta}(q, P)$ by invoking the (c, r) -kNN query. The answer can satisfy the distance criterion or the recall criterion according to the different return results of the (c, r) -kNN query.

Algorithm 2: Preprocessing Algorithm.

Input: a point set P
Output: a hierarchical Delaunay graph $HDG(P)$

```

1  $T \leftarrow \text{SplitTree}(P)$ ;
2 Modify  $T$  into a BMST;
3  $\text{ComputeSpheres}(T)$ ;
4  $\text{HierarchicalDelaunay}(T)$ ;
5 Procedure  $\text{SplitTree}(N)$ :
6   Conduct median split on  $N$  and generate two sets  $N_1$  and  $N_2$ ;
7    $T_1 \leftarrow \text{SplitTree}(N_1)$ ;
8    $T_2 \leftarrow \text{SplitTree}(N_2)$ ;
9   Let  $T_1$  be the left sub-tree of  $N$ , and  $T_2$  be the right sub-tree of  $N$ ;
10 end
11 Procedure  $\text{ComputeSpheres}(T)$ :
12   foreach  $N \in T$  do
13     Call  $\text{AMES}(N, 0.1)$  (Algorithm 1);
14   end
15 end
16 Procedure  $\text{HierarchicalDelaunay}(T)$ :
17   Let  $dl$  be the depth of the leaf node in  $T$ ;
18   for  $i = 0$  to  $dl$  do
19      $\text{Delaunay}(L_T(i))$  (Lemma 2.4);
20   end
21 end

```

Algorithm 3: Query.

Input: a query points q , a point set P , approximation factors $c > 1, \delta < 1$, and $HDG(P)$
Output: $k\text{ANN}_{c,\delta}(q, P)$

```

1  $N \leftarrow$  the root of  $HDG(P)$ ;
2 while  $|N| > 2k$  do
3    $Lc \leftarrow$  the left child of  $N$ ,  $Rc \leftarrow$  the right child of  $N$ ;
4   if  $D(q, \text{Cen}(Lc)) < D(q, \text{Cen}(Rc))$  then
5      $N \leftarrow Lc$ ;
6   else
7      $N \leftarrow Rc$ ;
8   end
9 end
10 while  $\exists N' \in \text{Nbr}(N)$  s.t.  $D(q, \text{Cen}(N')) < D(q, \text{Cen}(N))$  do
11    $N \leftarrow \arg \min_{N' \in \text{Nbr}(N)} \{D(q, \text{Cen}(N'))\}$ ;
12 end
13 for  $i = 0$  to  $\log_{\sqrt{c}} n + 2$  do
14   Invoke  $(\sqrt{c}, r)$ -kNN query where  $r = \frac{D(q, \text{Cen}(N)) + \text{Rad}(N)}{nc} \sqrt{c}^i$ ;
15   if the query returned a set  $\text{Res}$  then
16     return  $\text{Res}$  as the final result;
17   end
18 end

```

Algorithm 3 describes the above process in pseudo codes, where $\text{Cen}(N)$ and $\text{Rad}(N)$ are the center and radius of the AMES of node N , respectively.

4. Analysis

The analysis in this section will assume that the input point set P follows the Poisson Point Process.

4.1. Correctness

Lemma 4.1. *If the For-loop in Algorithm 3 terminates when $i = 0$, then the returned point set Res is a δ -kNN of q in P with at least $1 - e^{-\frac{n-k}{n^d}}$ probability.*

Proof. Let $R = D(q, \text{Cen}(N)) + \text{Rad}(N)$, $R_0 = R/n$, $t \in [0, k]$ be an integer. We define the following three events.

$$A = \{|P \cap S(q, R_0)| \geq k\}$$

$$B = \{|P \cap S(q, R_0)| \geq k + t\}$$

$$C = \{|\text{Res} \cap k\text{NN}(q, P)| \leq \delta \cdot k\}$$

The lemma states the situation that the algorithm terminates when $i = 0$, which implies that event A happens. Event C represents the situation that Res is a δ -kNN set. Then it is easy to see that the desired probability is in this lemma is $1 - \Pr[C \mid A]$. By the formula of conditional probability,

$$\Pr[C \mid A] = \Pr[C \mid B, A] \Pr[B \mid A] \leq \Pr[B \mid A].$$

Thus in the rest of the proof we focus on calculating $\Pr[B \mid A]$.

To calculate $\Pr[B \mid A]$ we need the probability that a single point p falls in $S(q, R_0)$. We have the following calculations.

$$\begin{aligned} \Pr[p \in S(q, R_0)] &= \Pr[p \in S(q, R_0) \mid p \in S(q, R)] \cdot \Pr[p \in S(q, R)] \\ &\leq \Pr[p \in S(q, R_0) \mid p \in S(q, R)] \\ &= 1/n^d \end{aligned} \quad (1)$$

The last equation is based on Lemma 2.2.

On the other hand, the number of points in $S(q, R)$ is at most n . Here we use the trivial upper bound of n since it is sufficient to the proof. Denote $P = \Pr[p \in S(q, R_0)]$, we have the following calculations.

$$\Pr[B \mid A] \leq \binom{n-k}{t} P^t (1-P)^{n-k-t} \leq e^{-P(n-k)} \frac{P^t (n-k)^t}{t!}$$

By the property of the Poisson Distribution, the above expression achieves the maximum when $t = \lfloor (n-k)P \rfloor = \lfloor (n-k)/n^d \rfloor = 0$. Thus we have

$$\Pr[B \mid A] \leq e^{-\frac{n-k}{n^d}}$$

Finally, combining the above analysis, we achieve the result that Res is a δ -kNN set with at least $1 - e^{-\frac{n-k}{n^d}}$ probability. $\square$

Lemma 4.2. *If the For-loop in Algorithm 3 terminates when $i > 0$, then the returned point set Res is a c -kNN of q in P .*

Proof. Let $R_{i-1} = \frac{D(q, Cen(N)) + Rad(N)}{nc} \sqrt{c}^{i-1}$ and $R_i = \frac{D(q, Cen(N)) + Rad(N)}{nc} \sqrt{c}^i$, which are the input parameters of the $(i-1)$ -th and i -th invocation of the (c, r) -kNN query, respectively. The lemma states the situation that the algorithm terminates at the i -th loop, which implies that the (c, r) -kNN query returns empty set in the $(i-1)$ -th loop. By Corollary 2.1, the number of points is less than k in the d -dimensional sphere $S(q, R_{i-1})$. Denote $T_k(q, P) = \max_{p \in kNN(q, P)} D(q, p)$, then it can be deduced that $T_k(q, P) \geq R_{i-1}$. On the other hand, the algorithm returns at the i -th loop, which implies that the (c, r) -kNN query returns a subset of $P \cap S(q, \sqrt{c}R_i)$. Thus we have $D(p, q) \leq \sqrt{c}R_i$ for each $p \in Res$. Finally, $D(q, p)/T_k \leq \sqrt{c}R_i/R_{i-1} = c$, which implies that the result set Res satisfies the definition of c -kNN. $\square$

Theorem 4.1. *The result of Algorithm 3 satisfies the requirement of $kNN_{c, \delta}(q, P)$ with at least $1 - e^{-\frac{n-k}{n^d}}$ probability.*

Proof. The result can be directly deduced by combining Lemma 4.1 and 4.2. $\square$

4.2. Complexities

For ease of understanding, we first analyze the complexity of the single steps in Algorithm 2 and 3.

Lemma 4.3. *The first step of Algorithm 2 takes $O(n \log n)$ time.*

Proof. The following recursion formula can be easily deduced from the pseudo codes of Algorithm 2.

$$T(n) = 2T(n/2) + O(n)$$

The $O(n)$ term comes from the time of splitting and computing the median. This recursion formula can be solved by standard process, and the result is $T(n) = O(n \log n)$. $\square$

Lemma 4.4. *The second step of Algorithm 2 takes $O(n \log n)$ time.*

Proof. According to Lemma 2.1, the time to compute the AMES of a point set is proportional to the number of points in this set. Thus we have the following recursion formula:

$$T(n) = 2T(n/2) + O(n)$$

The answer is also $T(n) = O(n \log n)$. $\square$

Lemma 4.5. *The third step of Algorithm 2 takes $O(n \log n)$ time.*

Proof. According to the definition and the building process of the HDG, there are 2^i nodes in the i -th layer. And by Lemma 2.4, the time to build the Delaunay triangulation is $O(n \log n)$. Thus, the time complexity of the third step is represented by the following equation:

$$\sum_{i=0}^{\log n} 2^i \log 2^i = \sum_{i=0}^{\log n} i \cdot 2^i$$

This result of this additive equation is $O(n \log n)$. $\square$

Lemma 4.6. *The navigating step (second step) in Algorithm 3 needs $O(dn^{1/d} \log n)$ time.*

Proof. From Lemma 2.5 we know that the navigating step goes through at most $O(n^{1/d})$ simplices. A d -dimensional simplex has $d - 1$ vertexes, and thus the number of nodes in the HDG visited by the navigation process is $O(dn^{1/d})$. While a node is visited, the process goes over the neighbors of this node, and from Lemma 2.3 we know that the expected maximum degree of each node is $O(\log n)$. Thus the total time of the navigating is $O(dn^{1/d} \log n)$. $\square$

Now we are ready to present the final results about the complexities.

Theorem 4.2. *The expected time complexity of Algorithm 2, which is the preprocessing time complexity, is $O(n \log n)$.*

Proof. The preprocessing consists of three steps, the time complexities of which are shown in Lemma 4.3, 4.4 and 4.5. Adding them and we get the desired conclusion. $\square$

Theorem 4.3. *The space complexity of Algorithm 2 is $O(n \log n)$.*

Proof. The space needed to store the HDG is the proportional to the number of the graph edges and tree edges. The number of graph edges connected to each node is $O(\log n)$ according to Lemma 2.3, and the number of tree edges is constant. On the other hand, the number of nodes in HDG is $O(n)$. Finally, we get the result that the space complexity is $O(n \log n)$. $\square$

Theorem 4.4. *The time complexity of Algorithm 3, which is the query complexity, is $O(dn^{1/d} \log n + kn^\rho \log n)$, where $\rho < 1$ is a constant.*

Proof. The time complexity of Algorithm 3 consists of three parts. For the first part, which is descending part, it is easy to see that the complexity is $O(\log(n/k))$. And the time complexity of the second part is already solved by Lemma 4.6, which is $O(dn^{1/d} \log n)$. The third part is to invoke the (c, r) -kNN query for $O(\log n)$ times. By Lemma 2.6 each invocation of (c, r) -kNN needs $O(kn^\rho)$ time where $\rho < 1$ is a constant. If we take this algorithm as a Turing reduction, then the time to invoke (c, r) -kNN query is constant. Thus the third step needs $O(kn^\rho \log n)$ time. Adding the three parts and the desired result is achieved. $\square$

4.3. Further discussions

After further analysis of the proposed algorithm, we found that the returned result may as well satisfy both of the two approximation criteria. We can not present a succinct form of the probability that the two criteria are simultaneously satisfied, but only give some hint to analyze the probability.

Proposition 4.1. *If the For-loop of Algorithm 3 terminates when $i = 0$, then the returned point set Res may as well be a c -kNN of q in P .*

Proof (Sketch). Let $R = D(q, \text{Cen}(N)) + \text{Rad}(N)$, $R_0 = R/n$, $t \in [0, k]$ be an integer. If the For-loop of Algorithm 3 terminates when $i = 0$, then by Corollary 2.1 it must be the case that $|P \cap S(q, R_0)| \geq k$. We define the following events:

$$A = \{\text{Res is } c\text{-kNN}\},$$

$$A_r = \{Res \subseteq S(q, r)\}.$$

Denote $T_k = \max_{p \in kNN(q, P)} D(p, q)$, and $S_r = |P \cap S(q, r)|$. We have the following integral formula representing the probability $\Pr[A]$.

$$\Pr[A] = \sum_{x=k}^x \int_0^{R_0} \Pr[A_{cr} | T_k = r, S_{R_0} = x] \cdot \Pr[T_k = r, S_{R_0} = x]$$

Since T_k is continuously distributed, we need the probability density function of $T_k(r)$, i.e., the p.d.f. of the k -th nearest neighbor distance. By Theorem 1 in [18], the p.d.f. is given in the following form:

$$f_{T_k}(r) = e^{-\lambda c_d r^d} \frac{d(\lambda c_d r^d)^k}{r k!}$$

where $c_d r^d$ is the volume of the d -dimensional ball with radius r .

On the other hand, we have

$$\begin{aligned} & \Pr[A_{cr} | T_k = r, S_{R_0} = x] \\ &= \sum_{y=k}^x \Pr[A_{cr} | S_{cr} = y, S_{R_0} = x, T_k = r] \cdot \Pr[S_{cr} = y | S_{R_0} = x, T_k = r] \end{aligned} \quad (2)$$

The first term can be calculated by classical probability model.

$$\Pr[A_{cr} | S_{cr} = y, S_{R_0} = x, S_{cr} = x, T_k = r] = \frac{\binom{y}{k}}{\binom{x}{k}}$$

The second term can be calculated by the property of Poisson Point Process. For a single point, the probability that it falls in $S(q, cr) \setminus S(q, r)$ is $P = \frac{(cr/R)^d - (r/R)^d}{1 - (r/R)^d}$. Thus,

$$\Pr[S_{cr} = y | S_{R_0} = x, T_k = r] = \binom{x-k}{y-k} P^{x-k} (1-P)^{x-y}$$

The rest of the work is to derive a lower bound of the above formula, and the sketch ends here. $\square$

Proposition 4.2. *If the For-loop of Algorithm 3 terminates when $i > 0$, then the returned point set Res may as well be a δ -kNN of q in P .*

Proof (sketch). Let $R = D(q, Cen(N)) + Rad(N)$, $R_0 = R/nc$, and $R_i = R_0 \cdot \sqrt{c}^i$ for $0 \leq i \leq 2 \log_c n + 2$. Denote $S(r) = |P \cap S(q, r)|$, and $T_k = \max_{p \in kNN(q, P)} D(p, q)$. If the For-loop of Algorithm 3 terminates at the i -th loop, then it must be the case that $S(R_{i-1}) \leq k$ and $S(R_{i+1}) \geq k$. We define the event $A = \{Res \text{ is } \delta\text{-kNN}\}$, and we have

$$\Pr[A] = \sum_{x=0}^k \sum_{y=k}^n \Pr[A | S(R_{i-1}) = x, S(R_{i+1}) = y] \Pr[S(R_{i-1}) = x, S(R_{i+1}) = y]$$

The first term can be calculated by classical probability model.

$$\Pr[A | S(R_{i-1}) = x, S(R_{i+1}) = y] = \frac{\binom{k}{\delta k} \binom{y-k}{k-\delta k}}{\binom{y}{k}}$$

The second term can be calculated by the property of Poisson Point Process. For a single point, the probability that it falls in $S(q, R_{i+1})$ is $P_1 = (\frac{c^{(i-1)/2}}{n})^d$. And the probability that a point falls in $S(q, R_{i-1})$ with condition that it is in $S(q, R_{i+1})$ is $P_2 = \frac{1}{c^d}$. Thus,

$$\Pr[S(R_{i-1}) = x, S(R_{i+1}) = y] = \binom{n}{y} P_1^y (1-P_1)^{n-y} \cdot \binom{y}{x} P_2^x (1-P_2)^{y-x}$$

The rest of the work is to derive a lower bound of the above formula, and the sketch ends here. $\square$

5. Conclusion

In this paper we proposed an algorithm for the approximate k-Nearest-Neighbors problem. We observed that there are two kinds of approximation criteria in the history of this research area, which is called the distance criterion and the recall criterion in this paper. We also observed that all existing works do not have full theoretical guarantees on these two criteria. We raised a new definition for the approximate k-Nearest-Neighbor problem which unifies the distance criterion and the recall criterion, and proposed an algorithm that solves the new problem. The result of the algorithm can satisfy at least one of the two criteria. In this extended paper we have found that there is a certain possibility that the returned result could satisfy both criteria. In our future work, we will try to devise new algorithms that can satisfy both of the criteria.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this manuscript.

Appendix A. Proof of Lemma 2.6

Proof. The algorithm for (c, r) -kNN is adapted from the standard LSH algorithm for (c, r) -NN. See [12] for more details. Briefly speaking, let $\mathcal{H}$ be a family of LSH functions, where p_1, p_2 are the characteristic probability. And let $\{h_i^j\}$ be a set of LSH functions uniformly drawn from $\mathcal{H}$, $1 \leq i \leq M$, $1 \leq j \leq L$, and $\mathcal{G}_j(p) = (h_1^j(p), \dots, h_M^j(p))$ be a composition of M LSH functions. The algorithm stores each element $p \in P$ in the hash bucket $\mathcal{G}_j(p)$, $1 \leq j \leq L$, and for the query point q , the algorithm scans the buckets $\mathcal{G}_j(q)$, $1 \leq j \leq L$, and collects the points in $S(q, cr) \cap \mathcal{G}_j(q)$. If the number of points collected reaches k , then the k points will be return as the result. If the algorithm has scanned $3L$ points before collecting enough points, it returns No .

Now we prove the algorithm succeeds with constant probability. The algorithm succeeds if the following two conditions are true. Here we call the points out side $S(q, cr)$ as *outer* points.

- A. If there exists $p_1, \dots, p_k \in B(q, r)$, then for each p_i there exists $\mathcal{G}_j$ such that $\mathcal{G}_j(p_i) = \mathcal{G}_j(q)$, and
- B. the algorithm encounters at most $3L$ outer points.

First we introduce the following two probabilities.

Let $P_1 = \Pr[\mathcal{G}_j(p') = \mathcal{G}_j(q) \mid D(p', q) \geq cr]$. Apparently $P_1 \leq p_2^K$. Then let $K = \log_{1/p_2} n \Rightarrow P_1 \leq \frac{1}{n}$. For all points outside $B(q, cr)$, the expected number of outer points satisfying $\mathcal{G}_j(p') = \mathcal{G}_j(q)$ for some j is at most $\frac{1}{n} \times n - 1$. Thus for all $1 \leq i \leq L$, the expected number of outer points satisfying $\mathcal{G}_j(p') = \mathcal{G}_j(q)$ is at most L .

By Markov's inequality,

$$\Pr[|\{ \mathcal{G}_j(p') = \mathcal{G}_j(q) \mid D(p', q) \geq cr \}| \geq 3L] \leq L/3L = \frac{1}{3}.$$

This is the possibility of scanning at least $3L$ outer points, which is the possibility that event B fails. Thus $\Pr[B] \geq \frac{2}{3}$.

On the other hand, let $P_2 = \Pr[\mathcal{G}_j(p) = \mathcal{G}_j(q) \mid D(p, q) \leq r]$. By setting $M = \log_{1/p_2} n$ we have the following derivations

$$P_2 \geq p_1^M = p_1^{\log_{1/p_2} n} = n^{-\frac{\log_{1/p_1} n}{\log_{1/p_2} n}} = n^{-\rho}$$

Setting $L = kn^\rho$, the possibility that there exists at least one $1 \leq j \leq L$ such that $\mathcal{G}_j(p) = \mathcal{G}_j(q)$ is at least

$$1 - (1 - P_2)^L \geq 1 - (1 - n^{-\rho})^{kn^\rho} \geq 1 - e^{-k}.$$

For all the k points, the possibility that p coincides with each of the k points, which is $\Pr[A]$, is that

$$\Pr[A] \geq (1 - e^{-k})^k \geq 1 - e^{-1}$$

The last inequality comes from the monotonicity of the function $(1 - e^{-x})^x$. Finally, the probability of condition A and B both succeed can be easily computed, which is constant probability. The details are omitted.

After all, we have proved that the algorithm can return the result of (c, r) -kNN with constant probability by setting $M = \log_{1/p_2} n$ and $L = kn^\rho$. Finally substituting the M and L with the proper values, we get the complexities of the algorithm, which is stated in Lemma 2.6. $\square$

References

- [1] Poisson point process, https://wikimili.com/en/Poisson_point_process.
- [2] G. Amato, C. Gennaro, P. Savino, MI-file: using inverted files for scalable approximate similarity search, *Multimed. Tools Appl.* 71 (3) (Aug 2014) 1333–1362.

- [3] A. Andoni, T. Laarhoven, I. Razenshteyn, E. Waingarten, Optimal hashing-based time-space trade-offs for approximate near neighbors, in: *Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms*, Jan 2017, pp. 47–66.
- [4] A. Andoni, I. Razenshteyn, Optimal data-dependent hashing for approximate near neighbors, in: *Proceedings of the Forty-Seventh Annual ACM on Symposium on Theory of Computing - STOC '15*, 2015, pp. 793–801.
- [5] M. Bădoiu, M. Bădoiu, K.L. Clarkson, K.L. Clarkson, Smaller core-sets for balls, in: *Proceedings of the Fourteenth Annual {ACM-SIAM} Symposium on Discrete Algorithms*, 2003, pp. 801–802.
- [6] J.L. Bentley, Multidimensional binary search trees used for associative searching, *Commun. ACM* 18 (9) (Sep 1975) 509–517.
- [7] M. Bern, D. Eppstein, F. Yao, The expected extremes in a Delaunay triangulation, *Int. J. Comput. Geom. Appl.* 01 (01) (Mar 1991) 79–91.
- [8] P. Bose, L. Devroye, On the stabbing number of a random Delaunay triangulation, *Comput. Geom. Theory Appl.* 36 (2) (2007) 89–105.
- [9] K. Buchin, W. Mulzer, Delaunay triangulations in $O(\text{sort}(n))$ time and more, in: *Proceedings - Annual IEEE Symposium on Foundations of Computer Science, FOCS*, vol. 5, 2009, pp. 139–148.
- [10] Z. Cai, D. Miao, Y. Li, Deletion propagation for multiple key preserving conjunctive queries: approximations and complexity, in: *35th IEEE International Conference on Data Engineering, ICDE 2019, Macao, China, April 8–11, 2019, IEEE*, 2019, pp. 506–517.
- [11] P.M.M. de Castro, O. Devillers, Simple and efficient distribution-sensitive point location in triangulations, in: *2011 Proceedings of the Thirteenth Workshop on Algorithm Engineering and Experiments (ALENEX)*, Jan 2011, pp. 127–138.
- [12] M. Datar, N. Immorlica, P. Indyk, V.S. Mirrokni, Locality-sensitive hashing scheme based on p-stable distributions, in: *Proceedings of the Twentieth Annual Symposium on Computational Geometry - SCG '04*, 2004, pp. 253–262.
- [13] O. Devillers, S. Pion, M. Teillaud, Walking in a triangulation, in: *Proceedings of the Seventeenth Annual Symposium on Computational Geometry - SCG '01*, ACM Press, New York, New York, USA, 2001, pp. 106–114.
- [14] A. Esuli, Use of permutation prefixes for efficient and scalable approximate similarity search, *Inf. Process. Manag.* 48 (5) (Sep 2012) 889–902.
- [15] J. Gan, J. Feng, Q. Fang, W. Ng, Locality-sensitive hashing scheme based on dynamic collision counting, in: *Proceedings of the 2012 International Conference on Management of Data - SIGMOD '12*, 2012, pp. 541–552.
- [16] J. Gao, H. Jagadish, B.C. Ooi, S. Wang, Selective hashing: closing the gap between radius search and k-NN search, in: *Proceedings of the 21st ACM SIGKDD International Conference on Knowledge Discovery and Data Mining - KDD '15*, 2015, pp. 349–358.
- [17] L. Gong, H. Wang, M. Ogihara, J. Xu, iDEC, *Proc. VLDB Endow.* 13 (9) (May 2020) 1483–1497.
- [18] M. Haenggi, On distances in uniformly random networks, *IEEE Trans. Inf. Theory* 51 (10) (2005) 3584–3586.
- [19] S. Har-Peled, A replacement for Voronoi diagrams of near linear size, in: *Proceedings 42nd IEEE Symposium on Foundations of Computer Science, IEEE*, 2001, pp. 94–103.
- [20] S. Har-Peled, P. Indyk, R. Motwani, Approximate nearest neighbor: towards removing the curse of dimensionality, *Theory Comput.* 8 (1) (2012) 321–350.
- [21] P. Indyk, R. Motwani, Approximate nearest neighbors: towards removing the curse of dimensionality, in: *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing - STOC '98*, 1998, pp. 604–613.
- [22] P.C. Lin, W.L. Zhao, Graph Based Nearest Neighbor Search: Promises and Failures, 2019, pp. 1–8.
- [23] K. Lu, H. Wang, W. Wang, M. Kudo, VHP, *Proc. VLDB Endow.* 13 (9) (May 2020) 1443–1455.
- [24] H. Ma, J. Li, An algorithm for reducing approximate nearest neighbor to approximate near neighbor with $O(\log n)$ query time, in: *12th International Conference on Combinatorial Optimization and Applications - COCOA 2018*, 2018, pp. 465–479.
- [25] Y. Malkov, A. Ponomarenko, A. Logvinov, V. Krylov, Approximate nearest neighbor algorithm based on navigable small world graphs, *Inf. Syst.* 45 (2014) 61–68.
- [26] D. Miao, Z. Cai, J. Li, On the complexity of bounded view propagation for conjunctive queries, *IEEE Trans. Knowl. Data Eng.* 30 (1) (2018) 115–127.
- [27] D. Miao, Z. Cai, J. Li, X. Gao, X. Liu, The computation of optimal subset repairs, *Proc. VLDB Endow.* 13 (11) (2020) 2061–2074.
- [28] D. Miao, J. Li, Z. Cai, Maximum reachability preserved graph cut, *Theor. Comput. Sci.* 840 (2020) 187–198.
- [29] D. Miao, J. Yu, Z. Cai, The hardness of resilience for nested aggregation query, *Theor. Comput. Sci.* 803 (2020) 152–159.
- [30] J.S. Mitchell, W. Mulzer, Proximity algorithms, in: *Handbook of Discrete and Computational Geometry*, third edition, 2017, pp. 849–874.
- [31] E.P. Mücke, I. Saías, B. Zhu, Fast randomized point location without preprocessing in two- and three-dimensional Delaunay triangulations, *Comput. Geom.* 12 (1–2) (1999) 63–83.
- [32] M. Muja, D.G. Lowe, Scalable nearest neighbor algorithms for high dimensional data, *IEEE Trans. Pattern Anal. Mach. Intell.* 36 (11) (2014) 2227–2240.
- [33] A. Ocsa, C. Bedregal, E. Cuadros-vargas, P.C. Society, A new approach for similarity queries using proximity graphs, in: *Simpósio Brasileiro de Banco de Dados*, 2007, pp. 131–142.
- [34] R. Paredes, E. Chávez, Using the k-nearest neighbor graph for proximity searching in metric spaces, in: *International Symposium on String Processing and Information Retrieval*, 2005, pp. 127–138.
- [35] Y. Sun, W. Wang, J. Qin, Y. Zhang, X. Lin, SRS: solving c-approximate nearest neighbor queries in high dimensional Euclidean space with a tiny index, *Proc. VLDB Endow.* 8 (2014) 1–12.
- [36] J. Wang, H.T. Shen, J. Song, J. Ji, Hashing for similarity search: a survey, *arXiv:1408.2927*, 2014.
- [37] R. Weber, H.J. Schek, S. Blott, A quantitative analysis and performance study for similarity-search methods in high-dimensional spaces, in: *Proceedings of 24th International Conference on Very Large Data Bases*, 1998, pp. 194–205.
- [38] P.N. Yianilos, Data structures and algorithms for nearest neighbor search in general metric spaces, in: *Proceedings of the Fourth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1993, pp. 311–321.
- [39] E.A. Yildirim, Two algorithms for the minimum enclosing ball problem, *SIAM J. Optim.* 19 (3) (2008) 1368–1391.